



Heavily based on material from:
Murach's PostgreSQL (3rd Edition), by Joel
Murach

Objectives

Applied

1. Create triggers that execute **before** or **after** an INSERT, UPDATE, or DELETE statement is executed for a table.

Knowledge

1. Describe how a row-level trigger works.
2. Describe how a trigger can be used to enforce data consistency or to insert rows into an audit table.

What is a Trigger?

A trigger is a named database object that is executed, or fired, **automatically** when a particular type of SQL statement is executed.

When using PostgreSQL, a trigger is fired when an INSERT, UPDATE, or DELETE statement is executed on a table.

Used by SQL programmers to prevent database errors when an action is executed.

Referential integrity means enforcing the rules of the relationships between the tables and the data in the tables.

In PostgreSQL, you have 2 ways to do this.

1. **Declarative Referential Integrity (DRI, primary/foreign keys), and**

2. Triggers.

Declarative referential integrity is a lot easier to implement and maintain, **but does not cover all the possible cases.**



A trigger is a specification that the database should **automatically execute** a particular function whenever a certain type of operation is performed.

Triggers can be attached to tables, views, and foreign tables.

In PostgreSQL, a **foreign table** is a database object which represents a table present on an external data source which is accessed by a foreign data wrapper (FDW).

- On tables, triggers can be defined to execute either before or after any **INSERT**, **UPDATE**, or **DELETE** operation, **either once per modified row, or once per SQL statement**.
- **UPDATE** triggers can moreover be set to fire only if certain columns are mentioned in the **SET** clause of the **UPDATE** statement.
- Triggers can also fire for **TRUNCATE** statements.
- If a trigger event occurs, the trigger's function is called at the appropriate time to handle the event.

- **On views**, triggers can be defined to execute instead of **INSERT**, **UPDATE**, or **DELETE** operations.
- Such **INSTEAD OF** triggers are fired once for each row that needs to be modified in the view.
- It is the responsibility of the trigger's function to perform the necessary modifications to the view's underlying base table(s) and, where appropriate, return the modified row as it will appear in the view.

- The **trigger function** must be **defined before the trigger itself can be created**.
- The **trigger function** **must be declared as a function** taking no arguments and **returning type trigger**.
 - The trigger function receives its input through a specially-passed TriggerData structure, not in the form of ordinary function arguments.
- Once a suitable trigger function has been created, the trigger is established with **CREATE TRIGGER**.
 - The same trigger function can be used for multiple triggers.

- PostgreSQL offers both per-row triggers and per-statement triggers.
- With a **per-row trigger**, the trigger function is invoked once for each row that is affected by the statement that fired the trigger.
- A **per-statement trigger** is invoked only once when an appropriate statement is executed, regardless of the number of rows affected by that statement.
 - A statement that affects zero rows will still result in the execution of any applicable per-statement triggers.
- These two types of triggers are sometimes called **row-level triggers** and **statement-level triggers**.
- Triggers on **TRUNCATE** may only be defined at statement level, not per-row.

- Triggers are also classified according to whether they fire **before**, **after**, or **instead** of the operation.
- **Statement-level BEFORE** triggers naturally fire before the statement starts to do anything, while statement-level **AFTER** triggers fire at the very end of the statement.
 - These types of triggers may be **defined on tables, views, or foreign tables**.
- **Row-level BEFORE** triggers fire immediately before a particular row is operated on, while row-level **AFTER** triggers fire at the end of the statement (but before any statement-level **AFTER** triggers).
 - These types of triggers may only be **defined on tables and foreign tables**, not views.
- **INSTEAD OF** triggers may **only be defined on views**, and only at row level.

When a PL/pgSQL function is called as a trigger, several special variables are created automatically in the top-level block. They are:

- **NEW** row: new database row for **INSERT/UPDATE** operations in **row-level triggers**. This variable is **null in statement-level triggers and for DELETE operations**.
- **OLD** row: old database row for **UPDATE/DELETE** operations in **row-level triggers**. This variable is **null in statement-level triggers and for INSERT operations**.
- **TG_OP** text: operation for which the trigger was fired: **INSERT, UPDATE, DELETE, or TRUNCATE**.

When	Event	Row-level	Statement-level
BEFORE	INSERT/UPDATE/ DELETE	Tables and foreign tables	Tables, views, and foreign tables
	TRUNCATE	—	Tables and foreign tables
AFTER	INSERT/UPDATE/ DELETE	Tables and foreign tables	Tables, views, and foreign tables
	TRUNCATE	—	Tables and foreign tables
INSTEAD OF	INSERT/UPDATE/ DELETE	Views	—
	TRUNCATE	—	—

```
CREATE TABLE triggers2.emp (  
    empname    text NOT NULL  
    , salary   integer  
);
```

```
CREATE TABLE triggers2.emp_audit(  
    operation      char(1)    NOT NULL  
    , stamp        timestamp  NOT NULL  
    , userid       text       NOT NULL  
    , old_empname  text  
    , new_empname  text       NOT NULL  
    , old_salary   integer  
    , new_salary   integer  
);
```

```
CREATE OR REPLACE FUNCTION triggers2.process_emp_audit()  
  RETURNS TRIGGER AS $$  
BEGIN  
  IF (TG_OP = 'DELETE') THEN  
    INSERT INTO triggers2.emp_audit  
      values('D', now(), current_user, NULL  
        , old.empname, old.salary, NULL);  
  ELSIF (TG_OP = 'UPDATE') THEN  
    INSERT INTO triggers2.emp_audit  
      values('U', now(), current_user, old.empname  
        , new.empname, old.salary, new.salary);  
  ELSIF (TG_OP = 'INSERT') THEN  
    INSERT INTO triggers2.emp_audit  
      values('I', now(), current_user, NULL  
        , new.empname, NULL, new.salary);  
  END IF;  
  RETURN NULL;  
  -- result is ignored since this is an AFTER trigger  
END;  
$$ LANGUAGE plpgsql;
```

```
CREATE TRIGGER emp_audit
  AFTER INSERT OR UPDATE OR DELETE
    ON triggers2.emp
  FOR EACH ROW
    EXECUTE FUNCTION
      triggers2.process_emp_audit();
```

```
insert into triggers2.emp
values
    ('Jesse Chaney', 10000)
;
insert into triggers2.emp
values
    ('Kevin McGrath', 15000)
    , ('Karla Fant', 20000)
;

select * from triggers2.emp_audit;
```

operation	stamp	userid	old_empname	new_emp...	old_salary	new_salary
I	2024-03-12 10:45:40.737407	win2024adb0058	<null>	Jesse Chaney	<null>	10000
I	2024-03-12 10:45:44.086042	win2024adb0058	<null>	Kevin McGrath	<null>	15000
I	2024-03-12 10:45:44.086042	win2024adb0058	<null>	Karla Fant	<null>	20000


```
update triggers2.emp
set salary = 11000
where empname = 'Jesse Chaney';

select * from triggers2.emp_audit;
```

operation	stamp	userid	old_empname	new_empname	old_salary	new_salary
I	2024-03-12 10:45:40.737407	win2024adb0058	<null>	Jesse Chaney	<null>	10000
I	2024-03-12 10:45:44.086042	win2024adb0058	<null>	Kevin McGrath	<null>	15000
I	2024-03-12 10:45:44.086042	win2024adb0058	<null>	Karla Fant	<null>	20000
U	2024-03-12 10:58:03.879074	win2024adb0058	Jesse Chaney	Jesse Chaney	10000	11000

```
update triggers2.emp
set empname = 'Jesse French'
where empname = 'Jesse Chaney';

select * from triggers2.emp_audit;
```

operation	stamp	userid	old_empname	new_empname	old_salary	new_salary
I	2024-03-12 10:45:40.737407	win2024adb0058	<null>	Jesse Chaney	<null>	10000
I	2024-03-12 10:45:44.086042	win2024adb0058	<null>	Kevin McGrath	<null>	15000
I	2024-03-12 10:45:44.086042	win2024adb0058	<null>	Karla Fant	<null>	20000
U	2024-03-12 10:58:03.879074	win2024adb0058	Jesse Chaney	Jesse Chaney	10000	11000
U	2024-03-12 11:00:23.718391	win2024adb0058	Jesse Chaney	Jesse French	11000	11000

```
delete from triggers2.emp
where empname = 'Jesse French';

select * from triggers2.emp_audit;
```

operation	stamp	userid	old_empname	new_empname	old_salary	new_salary
I	2024-03-12 10:45:40.737407	win2024adb0058	<null>	Jesse Chaney	<null>	10000
I	2024-03-12 10:45:44.086042	win2024adb0058	<null>	Kevin McGrath	<null>	15000
I	2024-03-12 10:45:44.086042	win2024adb0058	<null>	Karla Fant	<null>	20000
U	2024-03-12 10:58:03.879074	win2024adb0058	Jesse Chaney	Jesse Chaney	10000	11000
U	2024-03-12 11:00:23.718391	win2024adb0058	Jesse Chaney	Jesse French	11000	11000
D	2024-03-12 11:02:36.981846	win2024adb0058	<null>	Jesse French	11000	<null>

```
CREATE OR REPLACE FUNCTION triggers2.emp_salary_check()  
  RETURNS TRIGGER AS $$  
  BEGIN  
    if new.salary < 0 then  
      RAISE EXCEPTION  
        'salary cannot be negative';  
    end if;  
    RETURN new;  
  END;  
  $$  
LANGUAGE plpgsql;  
  
CREATE TRIGGER emp_salary_check  
  BEFORE INSERT OR UPDATE ON triggers2.emp  
  FOR EACH ROW EXECUTE FUNCTION  
    triggers2.emp_salary_check();
```

```
insert into triggers2.emp
values
    ('Jesse Chaney', -10000)
;

update triggers2.emp
set salary = -10000
where empname = 'Jesse Chaney';
```

```
[P0001] ERROR: salary cannot be negative
Where: PL/pgSQL function triggers2.emp_salary_check() line 4 at RAISE
```

```
CREATE OR REPLACE FUNCTION triggers2.emp_unique_check()  
  RETURNS TRIGGER AS $$  
  BEGIN  
    if new.empname in  
      (select empname from triggers2.emp  
       where empname = new.empname) then  
      raise 'duplicate empname';  
      RETURN NULL;  
    end if;  
    RETURN new;  
  END;  
  $$  
LANGUAGE plpgsql;  
  
CREATE TRIGGER emp_unique_check  
BEFORE INSERT OR UPDATE ON triggers2.emp  
  FOR EACH ROW EXECUTE FUNCTION  
    triggers2.emp_unique_check();
```

```
insert into triggers2.emp  
values  
    ( 'Jesse Chaney' , 10000 )  
;
```

```
[P0001] ERROR: duplicate empname  
Where: PL/pgSQL function triggers2.emp_unique_check() line 4 at RAISE
```

```
CREATE OR REPLACE FUNCTION triggers2.emp_truncate()  
  RETURNS TRIGGER AS $$  
  BEGIN  
      insert into triggers2.emp_audit  
      select 'T', now(), current_user, NULL  
        , empname, NULL, salary  
      from triggers2.emp;  
      RETURN NULL;  
  END;  
  $$  
LANGUAGE plpgsql;  
  
CREATE or replace TRIGGER emp_truncate  
BEFORE truncate ON triggers2.emp  
  FOR EACH STATEMENT EXECUTE FUNCTION  
    triggers2.emp_truncate();
```



```
truncate table triggers2.emp;
```

operation	stamp	userid	old_empname	new_empname	old_salary	new_salary
I	2024-03-12 14:58:04.389973	win2024adb0058	<null>	Jesse Chaney	<null>	10000
I	2024-03-12 14:58:07.658334	win2024adb0058	<null>	Kevin McGrath	<null>	15000
I	2024-03-12 14:58:07.658334	win2024adb0058	<null>	Karla Fant	<null>	20000
T	2024-03-12 15:00:01.450083	win2024adb0058	<null>	Jesse Chaney	<null>	10000
T	2024-03-12 15:00:01.450083	win2024adb0058	<null>	Kevin McGrath	<null>	15000
T	2024-03-12 15:00:01.450083	win2024adb0058	<null>	Karla Fant	<null>	20000